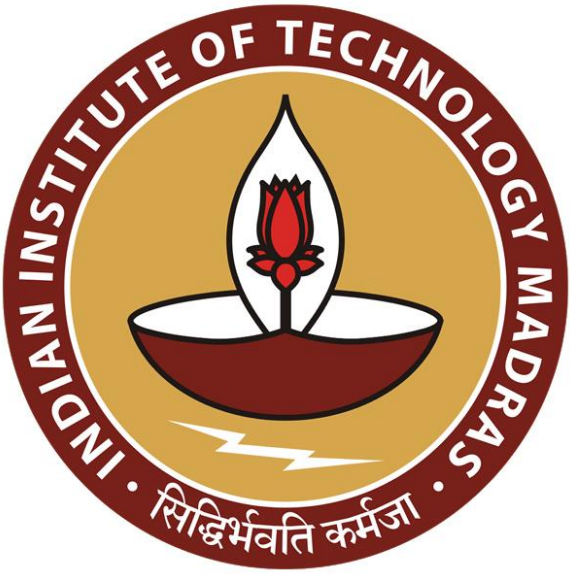


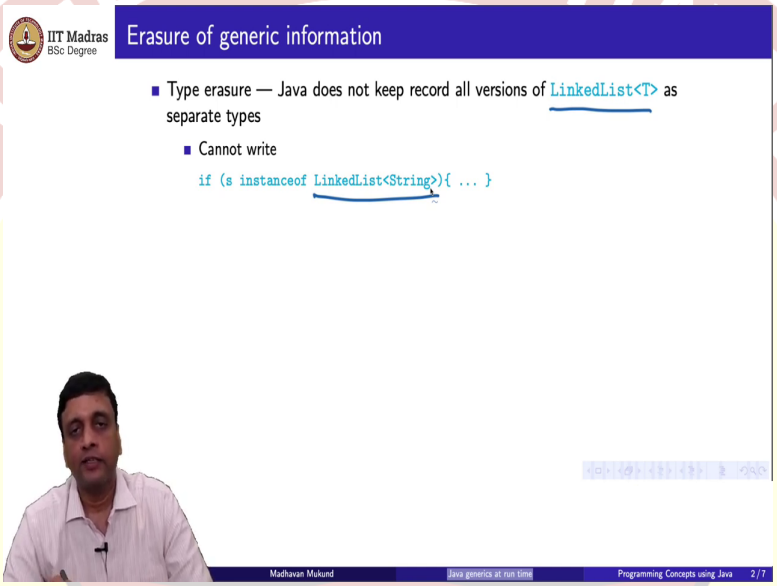
IIT Madras

ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Java Generics at Run Time

So, let us come back to Java Generics and try to understand how Java Generics work at runtime.

(Refer Slide Time: 0:20)

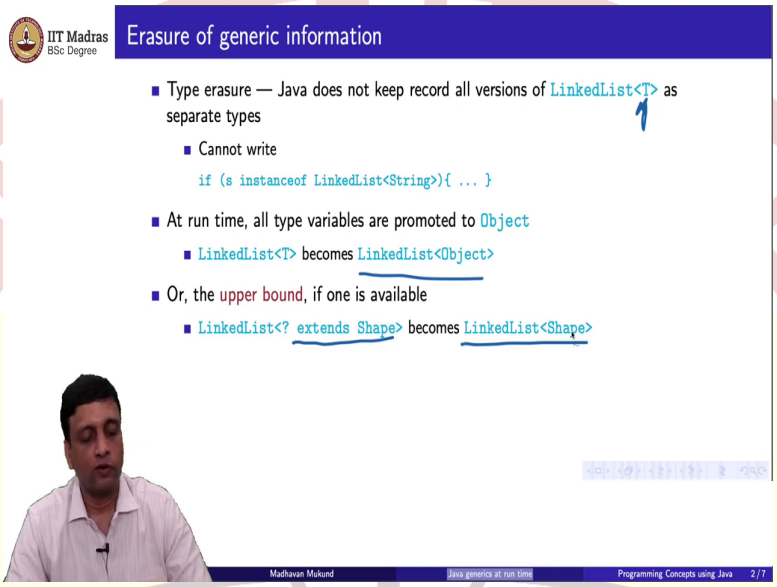


So, one important fact about Java generics is that Java does not keep track of the type at runtime. So, we saw earlier that there is no compatibility maintained between different instantiations of the type. So, we said that these are not covariant. So, basically, if I have a linked list of object, it is not a super type of a linked list of string. But in fact, it is more serious than that.

So, Java does not actually remember the type parameter at runtime. So, this means for instance, that so there is only in some sense when I write a parameterized class, it only compiles into one class, at runtime. So, it is not that I keep instantiating this class with different definitions each time I create a new instance of it. So, one of the consequences of this is that I cannot query whether something is an instance of a specific version of a class, if I have a linked list of an arbitrary type T, I cannot ask whether at a given point S is an instance of the linked list of string.

This is not available to me because Java does not have this information. It does not have a notion of linked lists of string except when I am writing my code when it is compiling. So, these type names and matching the type variables is only a compiler level thing. So, when the source code is checked, this is checked for compatibility. But at runtime, this information is somewhat curiously thrown away.

(Refer Slide Time: 1:47)



And the way it is thrown away is that internally, it actually kind of pretends what it does not allow otherwise, that everything is actually of the highest type. So, everything is compatible in the sense that internally, there is only one kind of generic linked list class. And the type of value that is associated with that linked list class is linked list of object. So, what this really means is that runtime, this type checking is all assumed to have been done correctly at compile time.

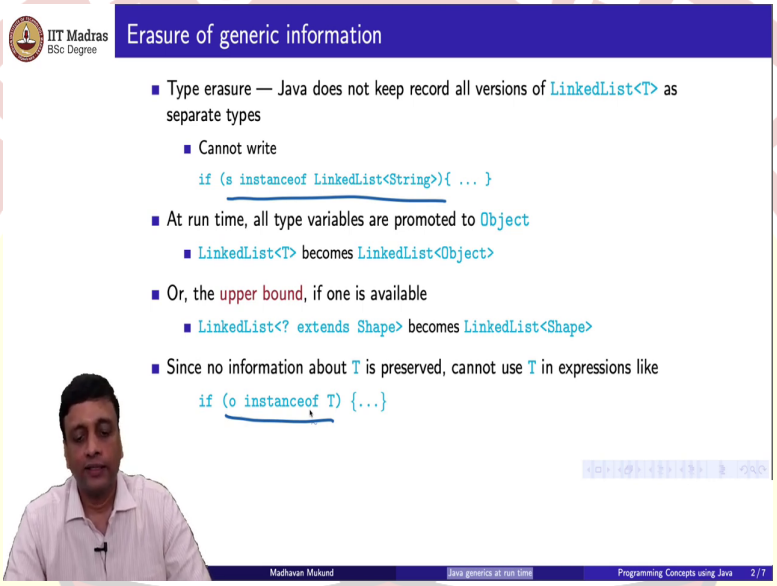
So, at runtime, it will freely allow you to do things. And it is the whole assumption is that everything that is typed unsafe should have been caught, because these syntactic constraints on the types and the instances were actually done at compile time. So, in general, if I have no constraint like if I just write T, then the assumption is that the internal object is as though it was LinkedList of object. So, T becomes promoted to the largest type possible in that type hierarchy, the highest type available is object.

But if I constrain t, if I say for instance, I have a list of something that extends shape, then I know that whatever list value I hold in this particular list, it can be no higher than shape in

my type hierarchy, because I have made that constraint at compile time. So, this would be associated inside at runtime with a linked list of type shape.

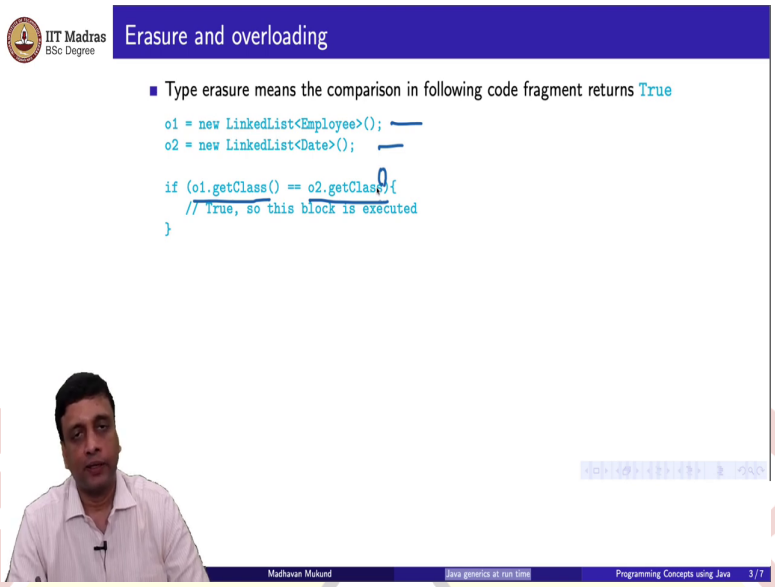
So, what Java does is at runtime, it converts an arbitrary kind of polymorphic data structure to one which is instantiated with the highest type compatible with the definition. if there is no constraint it goes to object there is a constraint, it goes to the what is called the upper bound of that (())(3:29).

(Refer Slide Time: 3:31)



So, another consequence is this we cannot ask this question. So, just like we cannot ask if it is a concrete instance of a particular type, we cannot also ask whether it is instance of a generic type because the generic type is also lost at runtime. So, this cannot be queried at runtime because we do not have information about T. So, syntactically, it means in our Java code, we cannot write instance of with anything to do with generics on the right-hand side either a concrete instance or a parameterized instance we cannot use this version of reflection in our Java code.

(Refer Slide Time: 4:03)

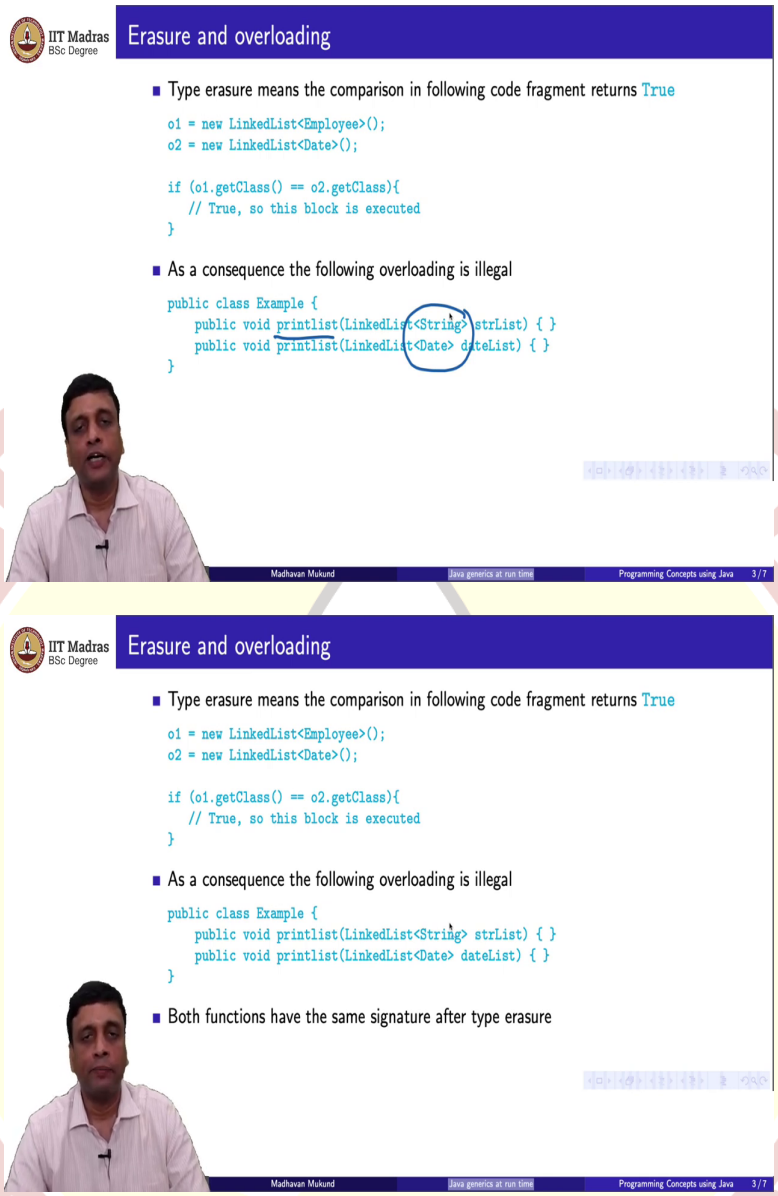


So, there are some other strange consequences of this. So, supposing I create two concrete objects by instantiating a parameterize class so here I have taken LinkedList and I have created an object one which is a LinkedList<Employee> and I created an object to which is a LinkedList<Date>. So, now, you might imagine that even though the type has been erased at runtime, these are created very clearly as 2 different types.

We can know that in compile time these two have no relationship with, so we cannot substitute a linked list of employee for a list of date vice versa. But if you are going to reflection, then the reflect class remember we can take an object and query its class and get its class information using the get class.

So, if I look at o1.getClass() and compare it to o2.getClass(). So, there should be a bracket here. If I do o2.getClass() and o1.getClass(), this is actually a class comparison returns true. So, this is somewhat paradoxical. So, I have created two parameterized versions of a parameterized type. At compile time these are incompatible. So, if I try to do this kind of equality assignment as passing a parameter anywhere Java will complain. But at runtime, I cannot distinguish them to the effect that reflection also cannot distinguish them.

(Refer Slide Time: 5:23)



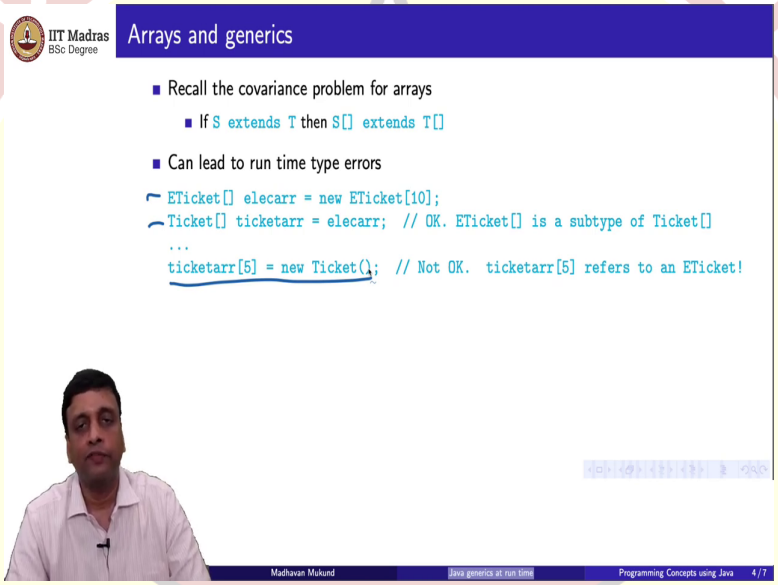
So, one of the consequences of this is that you cannot write functions which differ only in parameterize types. Remember, the Java allows overloading, you can use the same name with different signatures. And the signature for a Java function is a combination of the argument types and the return type.

So, we can write different versions with different arguments, different behavior, and distinguish which version we are using by changing the signature. But supposing the only change in the signature is to change the type parameter of a parameterized type. So, the first one tries to print a list of strings, and the second one tries to print a list of date. And perhaps

we want to print them in two different ways. Maybe the date has to be formatted in a particular format, where strings are just printed as they are.

Now, according to Java's type erasure, both LinkedList<String> and LinkedList<Date> will become essentially a linked list of objects, so they are the same type. So, this kind of overloading will not be allowed by the compiler, and it will be deemed to be illegal. So, both of them had the same signature after type erasure. So, this is, again, a consequence of the fact that the type information in a parameterized type and the generic type is erased at the time of running the programming.

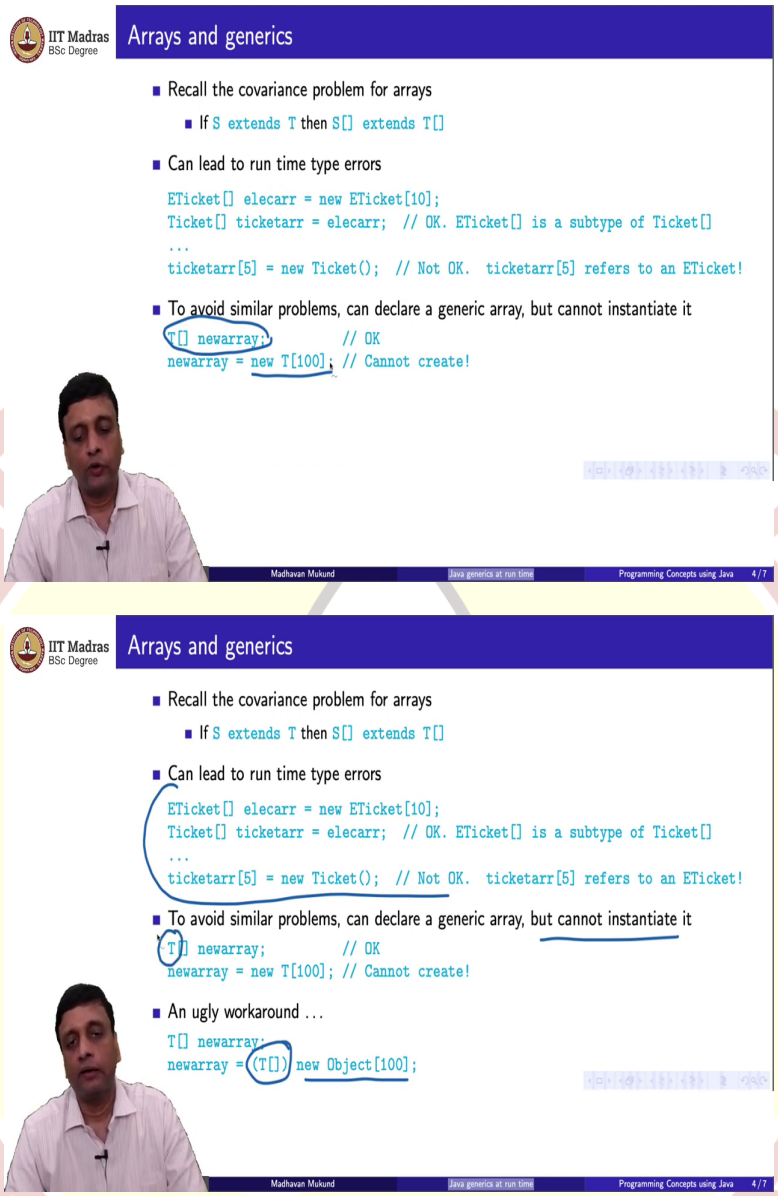
(Refer Slide Time: 6:32)



This also has an impact on interaction between arrays and generic. So, remember that we said that arrays were covariant, if S extends T, then an array of S extends an array of T, which seems somewhat natural to us. But this is not allowed for generics. So, for generics, substituting a super type for a subtype does not give you two compatible data structures.

So, what happens with arrays we saw was that we could have this kind of runtime type error, we take a super type ticket and make it point to an array of a subtype. And then we later try to replace an entry in that super type array by a super type object. And it says that this assignment is incompatible with the definition of the original array, which was of the subtype. So, this kind of problem we do not want to have.

(Refer Slide Time: 7:23)

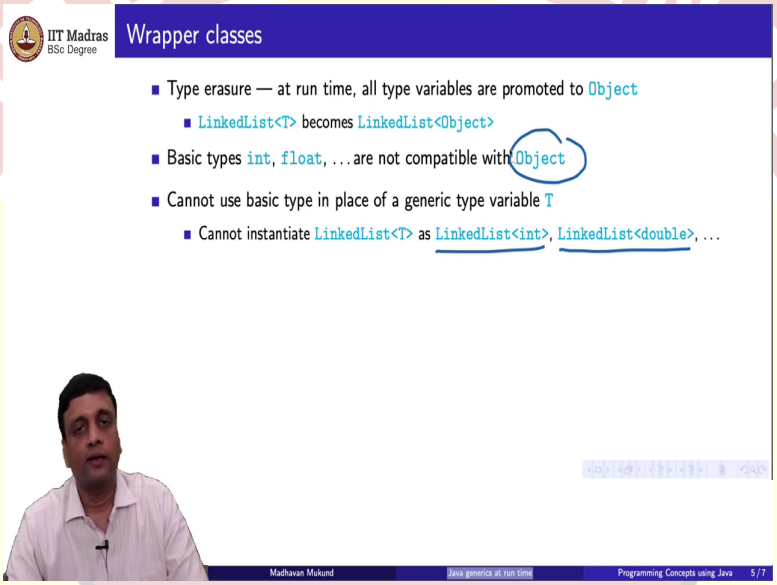


And so, Java does not allow us to actually instantiate arrays of generics. So, strangely, you can create a definition, so you can define a new array to be an array of some generic type t, but you cannot then instantiate it and say, give me a concrete array of size 100 of type T, this is not allowed.

What you can do is kind of work around it, and you can create a generic object and then we can type cast it, and then it is our responsibility as a programmer to make sure that at runtime this type class will work. So, this is the whole thing about typecasting, then, in some sense, it is a signal to the compiler, saying that, I am promising you that, I know that when this program runs, this type conversion will work.

And if you throw an error at that time, it is not your fault, it is more or less absolving the compiler of this problem. And the reason for not allowing this instantiation you can work it out for yourself, it will basically come down to the same thing that if I allow instantiation of arrays and if I erase information about the type of the array, then when I have a assignment later on, I could have a runtime error which the compiler should have caught but it is not able to catch because this type information was erased. So, that is why you cannot instantiate directly arrays of generics.

(Refer Slide Time: 8:43)



There is another small consequence about this type erasure. So, remember, we said that in general, all type variables will be promoted to object. So, a linked list of T, if I have no constraints on T will become a linked list of object. So, then whatever I store in that thing has to be in some sense compatible with storing an object.

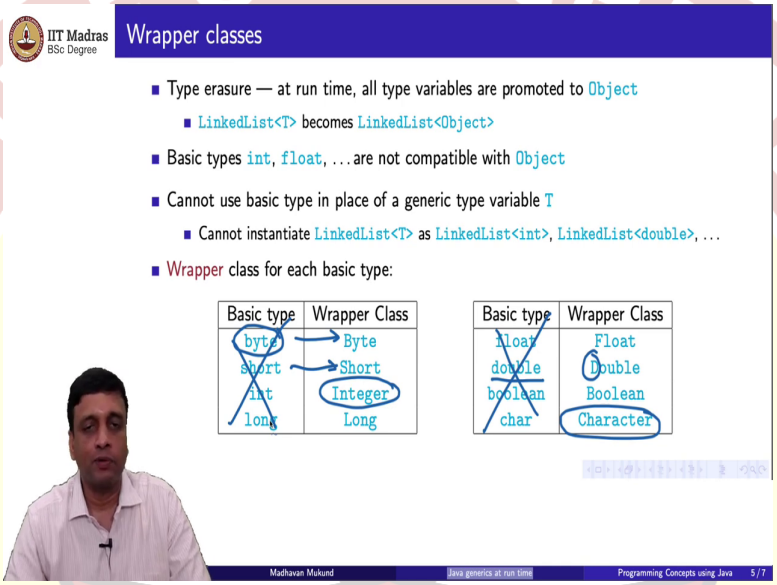
But we know that we have these built in types int, float, double car, boolean, which are not objects. So, I cannot take values of these simple types and use them where I am expecting something which is a real object. Now, one could argue that one should never have had built in types to begin with that everything should have been an object and this would have been really a very strict version of an object-oriented programming language.

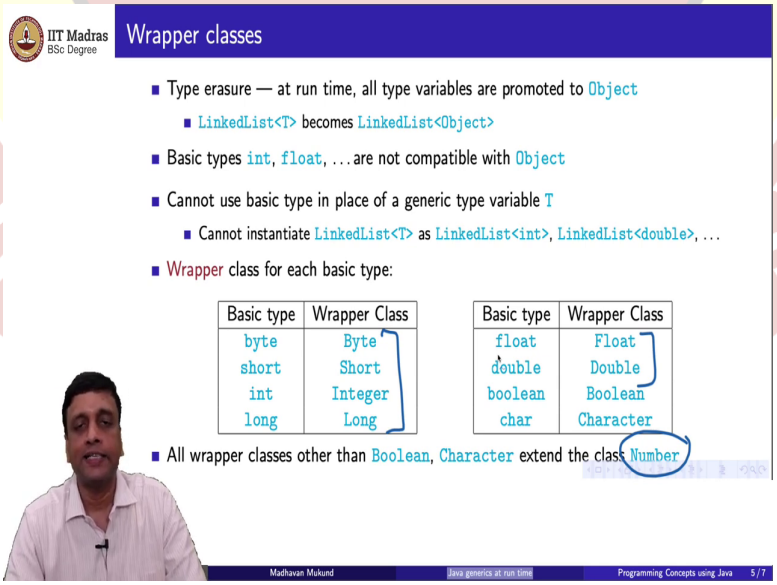
So, there are programming languages, which are object oriented, which do not have any built-in scalar types in some sense, everything is an object and is compatible with other objects. But unfortunately, Java has these two types of things. So, it has these eight primitive

types. And then it has this entire class hierarchy which is sitting under Object and forms a tree. So, how do we resolve this?

We cannot use now because of this type erasure, we cannot say linked list of small int and we cannot say linked list of small double, because a small double and a small int are these basic scalar types and they are not compatible with unfortunately this hierarchy of user defined types which sits under object.

(Refer Slide Time: 10:20)





So, in order to get around this kind of confusion, Java has to associate what is called a wrapper type with each built in type. So, with each basic type there is a wrapper class. So, byte is 1 byte of storage, it has a wrapper class, which is capital Byte, short is capital Short.

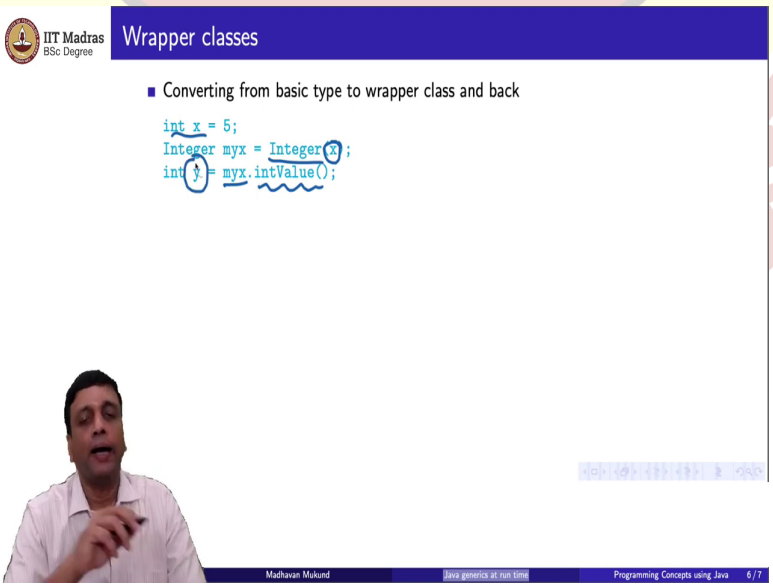
So, the only two names which are different are Integer and Character. So, in every other case, you take the primitive type name like double, and you just capitalize the first letter and you get the corresponding wrapper class.

So, what this actually amounts to is that these are the corresponding object compatible versions of these built in types. So, wherever I wanted to use an int, I could use an integer wherever I wanted to use a float, I could use a capital F Float. Now, as I said, if Java had had this thought at the beginning, they could have very well decided not to have these types at all and just have only the object types.

But then, perhaps in order to make the language more palatable and more, more like other languages like C and C++, they felt that they should retain some scalar types, but whatever the reason this decision to keep these two types leads to this unnecessary kind of dichotomy between basic types and wrapper classes. So, how do we use this?

Well, just before that, notice that all of these types other than Boolean and Character denotes some numeric quantity. So, inside the type hierarchy of Java, Java actually has a type called Number which is a super type of all these six numeric types. So, this is just something that you see occasionally you will look for a type that extends Number and what you mean is one of these wrapper classes which is a numeric type.

(Refer Slide Time: 11:58)



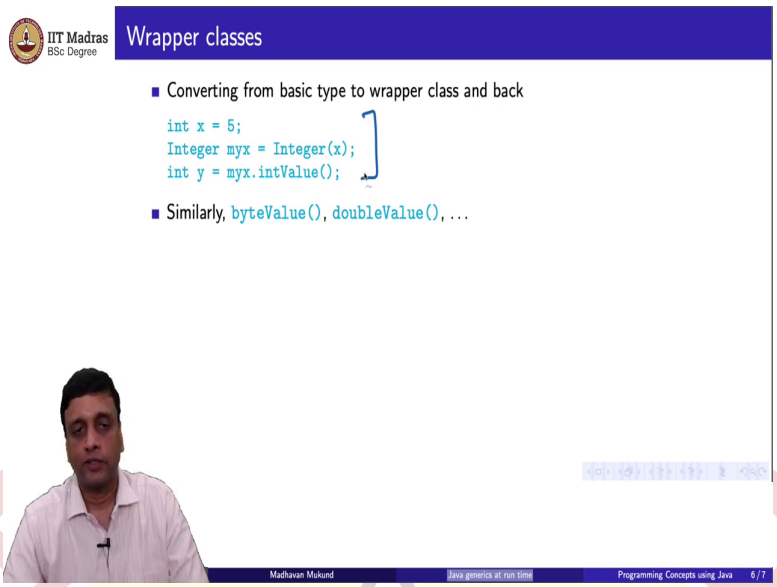
So, to convert back and forth, so, supposing we are working with int, then remember the corresponding wrapper class is called Integer. So, we start with a simple int, and assign it a

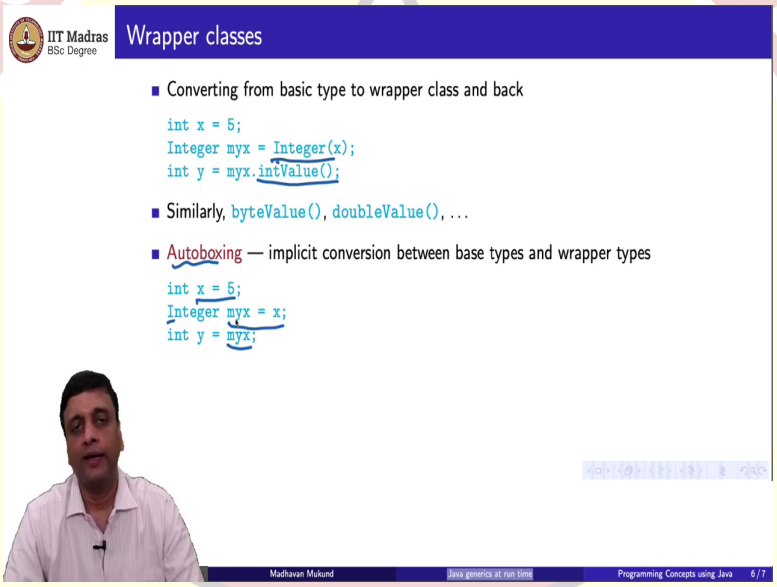
value 5 and now we want to create an Integer object which has the same value, then we use capital Integer as a constructor and pass it either an Integer value or a variable of type Integer as a value and it creates an integer object which stores that value.

So, the only purpose of a wrapper class, wrapper object is to store one value of that given type. So, an Integer object can store one Integer, a Double object can hold one double right and these are all immutable and final you cannot change anything and so on. So, you will have to create a new object you cannot take an Integer object and update it. We will have to create a new Integer object which has one more than Integer object. So, in many ways, these are just like scalar types should be, but they are just compatible with the object hierarchy.

So, anyway so you can take this scalar X and convert it to wrapper type integer by passing it to the Integer constructor, and then you can take the wrapper type and extract its underlying value using this particular function in this case called int value, and reassign it back to a scalar type. So, you can go from a scalar type to a wrapper type and come back. And this is for integer.

(Refer Slide Time: 13:23)





So, correspondingly, if the wrapper class had been a Byte, then you have a byte value function, it would have been Double, you have a double value function and so on. So, for each of these wrapper classes, you have a corresponding function which will extract the scalar or the simple type associated with the value stored inside that thing. Now, this of course, is rather tedious, because we will be forced to use these wrapper types wherever we have now parameterized classes and in particular, where we have these generic data structures, you want lists of x, or later on, we will see arrays and other things, which are parameterized.

And if everywhere, we have to go through this rigmarole of converting integers to; ints to capital integers, and then back, it will be a real nuisance. So, for this, Java actually has an automatic way of doing it, which is called Autoboxing. So, essentially, it allows you to freely

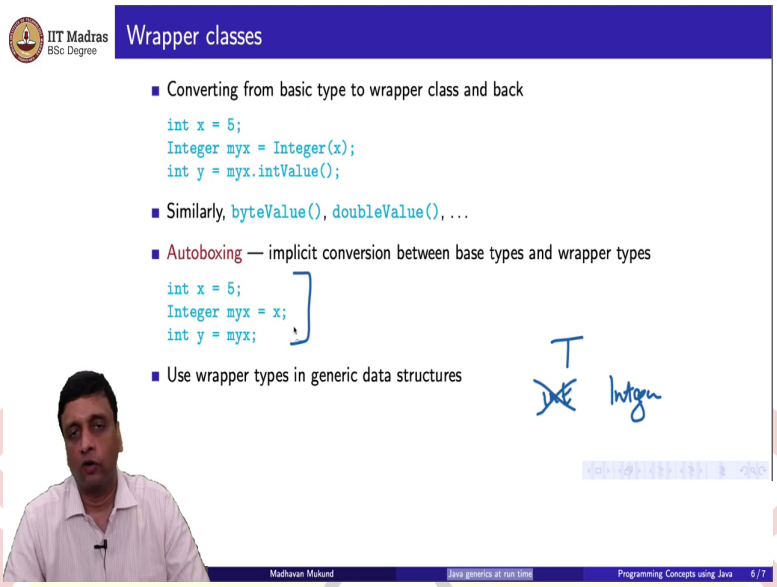
assign between these two types. You do not have to go around calling this constructor, you do not have to go around calling this function.

So, you can replace the code above by this much simpler code, which is you start with a scalar int x, you just assign it as though it was compatible with a capital integer. And as long as the types of the two sides are the same in the sense they are compatible that is the right hand side is a scalar version of the wrapper type on the left hand side, this will automatically create, so that is why it is called Autoboxing.

It is like you take a value and you wrap a box around it, so this wrapper is like you think of an object remember we should think of an object as a kind of black box, in which we can assign, put in data which we cannot see and then we can interact with this box through some designated public function. So, we are really taking the scalar integer and putting it into a box automatically.

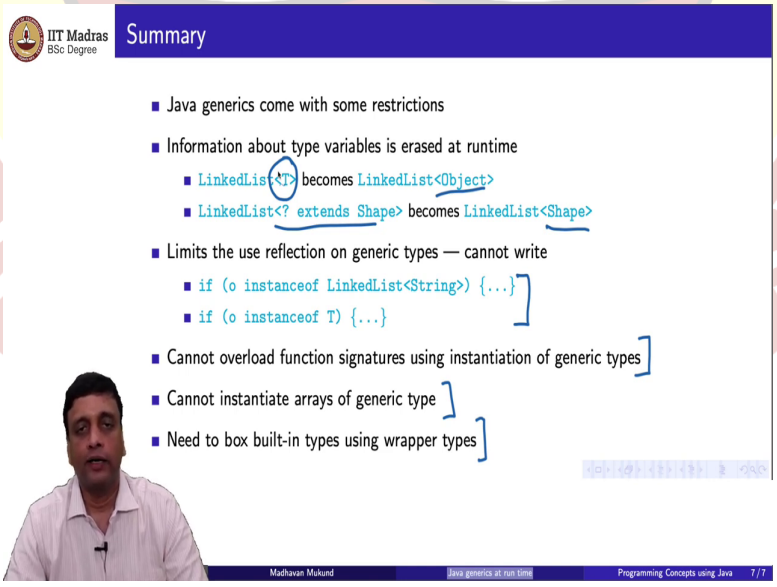
And similarly, this is unboxing, if you would like, we can take the value in the box and extract it and assign it to a scalar valuable without asking for the explicit function. So, so this actually makes it convenient to work, I mean except for typing this extra capital letter and this longer word, you can effectively use an integer wherever you use an int, and so on. So, it is much less painful than it would seem at the beginning, where you have this wrapper classes associated with built in types.

(Refer Slide Time: 15:33)



And we need wrapper classes, as I said, because in a generic data structure, where I see a type variable T, I cannot use int. So, this will not work, I have to use Integer. So, these are the situations where you will be obliged to use the wrapper classes. But fortunately, thanks to Autoboxing, you do not have to really do any extra work in order to use them, you can use them pretty much like you use the same scalar code in all your Java code.

(Refer Slide Time: 15:57)



So, what we saw is that these, the introduction of generics in Java, which brings in flexibility in terms of the structural polymorphism, comes with some constraints. So, the first thing is that Java erases information about type variables at runtime. And part of this erasure was

done essentially because it was added on. So, generics were added on to Java. So, there had to be a stage in between where they were Java code written without generic type variables and Java code written with generic type variables.

And you need to make sure that code of one type could call code of the other type and vice versa. So, for mainly for that reason, Java has a strange way of dealing with generic type information by just throwing it away and making it, promoting it in some sense to object unless you have a constraint in which case it promotes it to the upper bound of the constraint. So, this is really a compatibility issue.

So, when you add something to an existing language, and people have already developed code in that language, it is not very convenient to expect everybody to go back and rewrite all their code to be compatible with the change. So, the change has to then be somehow massaged into a form which is acceptable and this leads to these kinds of maybe counterintuitive consequences.

So, because of this erasure of types, made we cannot use these parameterized types or type variables in this reflection type of statements, where we query information about the type. Also, we cannot create overloaded functions, which differ only in the generic instantiation. So, if we instantiate a generic type with two different concrete object types, we cannot use that as a way of distinguishing between two functions, because they are all compiled into the same type, erased type by Java.

Then also because of this covariance problem, Java does not allow you to instantiate arrays of the generic type. So, if you want to mix generics and arrays, you have to go through a little bit of contraction. And finally, we have this problem that the built-in scalar types, the Numbers and the Boolean and the Characters that we introduced right at the beginning, which are compatible with more conventional programming languages are not in this object hierarchy.

So, they cannot be substituted for T because when I erase the type, everything that I store must be compatible with object or with some object in the class and therefore I cannot make an integer scalar int or a scalar double or a scalar boolean compatible with that. So, we have got these wrapper types, which in some sense, promote every scalar value to an equivalent

value which is compatible with our object hierarchy. But fortunately, because of this, Autoboxing, this conversion back and forth, is less painful than it appears at first sight.